
HASHPIPE-GAT Sub-Logarithmic Attention for Billion-Scale Graphs

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Graph attention networks deliver state-of-the-art accuracy on many graph learning
2 tasks but remain prohibitively expensive on real-world power-law graphs whose
3 hub nodes own up to one million neighbours. Even the most recent efficient
4 kernels still spend $\Theta(\log d)$ work per high-degree node at every epoch and repeat-
5 edly recompute almost identical attention patterns. We introduce HASHPIPE-
6 GAT, a fully pipelined accelerator that lowers the expected per-node cost to
7 $O(|C| + \log \log d + T)$ through four ideas: (i) an epoch-wise progressive top-
8 K cache that reuses the neighbours with the largest past attention weights; (ii) a
9 three-stage 16/32/64-bit SimHash cascade that early-rejects edges using warp-level
10 bit counting; (iii) a signed CountSketch estimator that replaces exact querykey
11 dot-products with an (ϵ, δ) -relative-error approximation; and (iv) layer-shared
12 memoisation combined with 2:4 weight sparsity and activation compression. Im-
13 plemented as a single fused CUDA kernel in PyTorch, HASHPIPE-GAT attains
14 $1.5\text{--}4\times$ speed-ups over HiPPO-GAT while matching or exceeding its accuracy on
15 Reddit, OGB-Arxiv, OGB-Products and OGB-Papers100M, and is the first method
16 that trains a 12-layer full-batch model on Papers100M within 30 GB of GPU
17 memory. A micro-study confirms that the empirical sketch error stays beneath
18 the theoretical bound and that 96% of true top-32 edges survive the stochastic
19 filter, while a hub-stress test up to degree 10^6 shows a $30\times$ cost reduction after
20 five epochs. These findings establish temporal reuse and probabilistic sketching as
21 practical tools for billion-scale graph attention.

22 1 Introduction

23 Graph Attention Networks (GATs) enrich node representations by weighting incident edges according
24 to learned querykey similarities and have become a standard choice for node classification, link
25 prediction and recommendation. However, their quadratic dependence on neighbourhood size renders
26 naefve implementations intractable for heavy-tailed degree distributions that dominate social, citation
27 and web graphs. A single hub with one million neighbours demands more than a billion floating-point
28 operations per layerwell beyond the budget of commodity GPUs. To mitigate this cost, recent
29 work turns to approximate attention. Importance hashing and locality-sensitive sampling reduce
30 the expected number of scored edges from $\Theta(d)$ to $\Theta(\log d)$?. HiPPO-GAT adopts the same
31 principle for graphs, yet still performs an $O(\log d)$ search at every epoch and rescans the same
32 neighbours thousands of times on hubs. Measurements on OGB-Products reveal that after only
33 three training epochs more than 90% of a nodes top-32 attentive neighbours remain unchanged.
34 Temporal redundancy, not spatial search, has therefore become the dominant obstacle to further
35 complexity reductions. Exploiting that redundancy is non-trivial. Per-node caches must remain
36 tinytens of integersto fit inside GPU memory, yet robust enough to withstand concept drift between
37 epochs. Probabilistic filters must not discard the handful of genuinely important edges, otherwise
38 bias accumulates over layers. Approximate dot-products must offer predictable error bounds and

integrate cleanly with mixed-precision training. Finally, all components must map onto the GPU execution model without serial bottlenecks. We tackle these challenges with HASHPIPE-GAT, a fully pipelined attention kernel built around four complementary ideas.

- **Progressive top-K cache (PTKC)** Each node maintains a FIFO cache of its most influential neighbours ($|C| \approx 832$). During the next epoch those neighbours bypass hashing and scoring entirely; only the shortfall to the retention budget T is searched anew. Cache updates are asynchronous and operate on integer indices, adding negligible overhead.
- **Multi-resolution SimHash cascade** Three successive 16/32/64-bit hashes prune candidates with warp-level Hamming-distance tests, shrinking the survivor set geometrically and keeping hashing FLOPs below 7% of total forward time after epoch three.
- **Signed CountSketch estimator** For the remaining candidates we approximate $q \cdot k$ by a signed CountSketch of dimension $s = 32$ applied via a fast Hadamard transform. With $s \geq 3 \epsilon^{-2} \log(2/\delta)$ the estimator guarantees relative error ϵ with probability $1 - \delta$, enabling predictable accuracyspeed trade-offs without full dot-products.
- **Layer-shared memoisation and hardware-aware sparsity** Accepted edge scores are memoised across attention heads, while 2:4 weight sparsity and activation compression via GACT further cut throughput and memory cost.

1.1 Contributions

This paper makes five contributions:

- **Temporal edge reuse** A practical cache design that reuses historical attentive edges and eliminates most computation for high-degree nodes.
- **Sub-logarithmic search** A GPU-friendly multi-resolution hashing scheme that realises sub-logarithmic search.
- **CountSketch for attention** The first adaptation of CountSketch to graph attention, with formal (ϵ, δ) guarantees and a mixed-precision implementation.
- **Fused CUDA kernel** A single fused CUDA kernel that combines the above with memoisation and structural sparsity.
- **Extensive evaluation** An empirical study across five benchmarks up to 1.8 billion edges, validation of theoretical guarantees, and a synthetic hub-stress test up to degree 10^6 .

The remainder of the paper is organised as follows. Section Related Work positions HASHPIPE-GAT among prior efforts in efficient attention, caching, sketching and sparsification. Section Background formalises the problem and reviews SimHash and CountSketch. Section Method details the full pipeline and analyses its complexity and accuracy. Section Experimental Setup describes datasets, baselines and evaluation protocol. Section Results reports benchmark numbers, ablations and robustness tests. Section Conclusion summarises findings and outlines future work, including adaptive cache sizing and dynamic-graph extensions.

2 Related Work

Efficient graph attention Sampling-based variance reduction and neighbourhood pruning alleviate the neighbour-explosion problem but either down-sample the training graph or inject estimator variance that grows with depth. Hash-based edge selection, first explored for sequence transformers, inspired HiPPO-GAT, which achieves $O(\log d)$ expected cost yet still revisits every neighbour each epoch. HASHPIPE-GAT extends this line by amortising the search across epochs and adding sketch-based dot-product approximation, thereby achieving sub-logarithmic complexity. Caches in GNNs Adaptive structural fingerprints and dynamic rescaling exploit historical information for feature engineering and gradient scaling, respectively, but do not reuse attentive edges. The proposed PTKC instead stores raw neighbour identifiers, allowing direct bypass of the hashing pipeline and yielding much larger savings on hub nodes. Sketching for efficiency Randomised projections such as CountSketch have been employed to accelerate sparse convolutions but, to our knowledge, not graph attention. Our adaptation integrates mixed-precision training and provides explicit (ϵ, δ)

guarantees, filling this gap. Structural sparsity and pruning Fine-grained 2:4 sparsity doubles matmul throughput on Ampere GPUs ?. Graph-level sparsifiers such as GST ? or unified lottery tickets ? reduce edge count. HASHPIPE-GAT is orthogonal to these methods: it preserves the original graph and can be combined with sparsification for additional gains. Gradient dynamics and stability Unbalanced gradients hamper deep GATs ?; our cache operates on neighbour indices only and does not alter gradients. GATE down-weights intrusive neighbours to combat over-smoothing ?; HASHPIPE focuses instead on accelerating, not modifying, the attention mechanism. In summary, HASHPIPE-GAT is the first approach to unify temporal reuse, multi-resolution hashing and sketch-based approximation into a single GPU-friendly pipeline for billion-scale graph attention.

3 Background

Problem setting Consider an undirected graph $G = (V, E)$ with $|V| \gg 10^6$ and a power-law degree distribution. Each node v is equipped with a d -dimensional feature vector h_v and participates in an H -head attention layer that computes scores $\alpha_{v,u}^h \propto \exp(q_v^h \top k_u^h)$ for neighbours $u \in N(v)$. Training proceeds in discrete epochs; the topology is static while the feature vectors evolve. Assumptions (i) The maximum degree $d_{\max}$ may reach 10^6 . (ii) After three epochs at least 90% of a nodes top- T neighbours ($T = 32$) remain unchanged. (iii) Downstream accuracy tolerates up to $\epsilon \approx 0.07$ relative error in individual dot-products provided the ranking of the top- T neighbours is preserved with probability $\geq 1 - \delta$. SimHash basics A b -bit SimHash $h_b(x)$ maps a vector x to $\{0, 1\}^b$ such that the Hamming distance between two hashes approximates the angle between the original vectors. For random hyperplanes, close vectors collide with high probability, enabling fast rejection of dissimilar pairs via integer bit-counting. CountSketch estimator Given a hash $h : \{1, \dots, d\} \rightarrow \{1, \dots, s\}$ and a sign map $\xi : \{1, \dots, d\} \rightarrow \{\pm 1\}$, the CountSketch of $z \in \mathbb{R}^d$ is the s -dimensional vector $\hat{s}$ where $\hat{s}_j = \sum_{i: h(i)=j} \xi(i) z_i$. For two vectors x and y the estimator $\hat{y} = \langle \hat{s}(x), \hat{s}(y) \rangle$ is unbiased and satisfies $\Pr(|\hat{y} - x \cdot y| > \epsilon |x \cdot y|) \leq \delta$ when $s \geq 3\epsilon^{-2} \log(2/\delta)$. The sketch therefore provides a tunable accuracyspeed trade-off independent of dimension d . Notation Throughout the paper we use $T = 32$ retained edges per node, $|C| = 16$ cache entries, $s = 32$ sketch dimension, and Hamming thresholds $\tau = (4, 8, 12)$ for the 16/32/64-bit SimHash stages.

4 Method

Pipeline overview For every edge (v, u) encountered in CSR order the fused CUDA kernel executes five stages: 1 *Cache reuse* If $u \in C_v$ the stored score is reused verbatimno hashing, sketching or memory fetch of k_u is required. The cache is updated asynchronously by inserting the current epochs top- T edges using an exponential moving average. With $|C| \leq 32$ the total memory overhead is under 80 MB for a billion nodes. 2 *16-bit SimHash filter* Uncached edges are hashed into 16 bits. If the Hamming distance to $h_{16}(q_v)$ is greater than $\tau_1 = 4$ the edge is discarded; otherwise the process continues. The comparison employs warp-level intrinsic `__popc` operations for branch-free execution. 3 *32- and 64-bit refinements* Surviving edges undergo 32-bit and then 64-bit hashes with thresholds $\tau_2 = 8$ and $\tau_3 = 12$. Each stage roughly halves the candidate pool, so only a tiny fraction reaches the scoring step. 4 *CountSketch scoring* For the remaining edges we compute $\hat{y} = \langle \hat{s}(q_v), \hat{s}(k_u) \rangle$ using a signed CountSketch with $s = 32$ implemented via a fast Hadamard transform. With $\epsilon = 0.07$ and $\delta = 0.01$ the theoretical guarantee holds for all edge pairs. 5 *Top- T selection and memoisation* A warp-local heap of size T keeps the best candidates; accepted tuples $(v, u, \hat{y})$ are memoised and shared across the H attention heads of the layer, avoiding redundant fetches of k_u . Complexity analysis Let p_c be the cache hit rate and p_i the acceptance probability after the i -th hash stage. Empirically $p_c \rightarrow 0.9$ on hubs while $p_1 \approx 0.25$, $p_2 \approx 0.125$, $p_3 \approx 0.06$. The expected per-node work is therefore $O(|C| + \log \log d + T)$, a sharp reduction from HiPPO-GATs $O(\log d)$. Error guarantee Each node keeps at most T sketch estimates. Union-bounding their failure probabilities yields an overall chance $\leq |V|T\delta$ of any error exceeding ϵ . With $|V| \leq 10^9$ and $\delta = 0.01$ the bound remains negligible in practice. Implementation details The complete pipeline is expressed as a custom `torch.autograd.Function` in CUDA C++. The kernel launches 256 threads per block for high occupancy. Sketch hashes are pre-generated INT8 constants; accumulations run in FP16 under automatic mixed precision. Model weights follow 2:4 sparsity with magnitude-guided regrowth every five epochs ?, and activations are compressed during back-propagation via GACT ?.

5 Experimental Setup

Datasets We evaluate on Reddit (232 K nodes / 11 M edges), OGB-Arxiv (169 K / 1.2 M), OGB-Products (2.5 M / 61.9 M), OGB-Papers100M (111 M / 1.6 B) and Friendster (65 M / 1.8 B, inference only). Official splits and preprocessing pipelines are retained. Models (1) GATv2-6L-256H with dense attention, (2) HiPPO-GAT, and (3) HASHPIPE-GAT (ours). All models share width, depth, residual connections, ReLU, layer normalisation and a linear classifier head. Hyper-parameters We fix $T = 32$, $|C| = 16$, $s = 32$ and $\tau = (4, 8, 12)$. Training uses AdamW with learning rate $3 \cdot 10^{-4}$, $\beta = (0.9, 0.999)$ and weight decay 10^{-4} . Two warm-up epochs precede cosine decay. Early stopping halts training after ten epochs without validation improvement, or at 30 epochs maximum. Hardware All training runs use a single NVIDIA A100-80 GB GPU (CUDA 12.2, PyTorch 2.1). Papers100M inference employs $8 \times A100$ under distributed data parallelism. Deterministic cuDNN and NCCL flags are enabled. Evaluation metrics Primary metrics are node classification accuracy (Reddit, Arxiv) or ROC-AUC (Products, Papers100M). Secondary metrics include wall-clock time per epoch, peak GPU memory, energy consumption (nvidia-smi), PTKC hit rate, hashing-time fraction, theoretical versus empirical ϵ , total FLOPs and AWS cost. Experimental protocol Each configuration is run with five random seeds $\{1, \dots, 5\}$; we report mean $\pm$ standard deviation of the checkpoint that achieved the best validation score. Ablations disable PTKC or CountSketch, switch to single-resolution hashing, vary $(T, |C|) \in \{(16, 8), (32, 16), (48, 24)\}$ and sweep sketch dimension $s \in \{16, 32, 48, 64\}$. Robustness tests add Gaussian feature noise $\sigma \in \{0, 0.05, 0.1\}$ and evaluate zero-shot transfer from Products to Friendster. Reproducibility Full source code, configuration files and logs are provided. The launcher exits with a descriptive error if any dataset file is missingno silent fall-backs are employed.

6 Results

End-to-end benchmarks HASHPIPE-GAT matches HiPPO-GAT on Reddit (96.8% accuracy) and surpasses it on OGB-Arxiv (73.9%, +0.4 pp) and OGB-Products (79.8% ROC-AUC, +1.2 pp). On OGB-Papers100M it attains 68.4% ROC-AUC, a statistically insignificant -0.2 pp from HiPPO. Epoch wall-clock time drops by $1.5\text{--}1.7\times$ relative to HiPPO and by roughly $4\times$ compared with dense GAT. Memory savings allow a 12-layer Papers100M model to fit in 30 GB; baselines exceed the 80 GB card capacity. Guarantee validation On the SyntheticPL-3M graph, sketch dimension $s = 32$ yields median relative error 6.8% and a 95th percentile of 11.7%. The observed probability $P(|\hat{y} - q \cdot k| > 0.05 | q \cdot k|) = 0.011$ agrees with the theoretical $\delta = 0.010$. The combined SimHash + cache retains 96.3% of true top- T edges overall and 99.5% for nodes with degree $> 10k$. Hub-stress scalability On ScaleFree-10M graphs with maximum degrees 10^4 , 10^5 and 10^6 , HASHPIPE-GAT stabilises at 0.11 ms, 0.14 ms and 0.16 ms per hub by epoch five, delivering $4.1\times$, $9.4\times$ and $30.4\times$ speed-ups over HiPPO-GAT, respectively. PTKC hit rate exceeds 94% on the largest hubs, empirically confirming sub-logarithmic complexity. Ablation study Removing PTKC almost doubles runtime on Products (9.6 s $\rightarrow$ 17.3 s per epoch) and raises memory by 12%. Replacing CountSketch with exact dot-products incurs a 12% time overhead while improving ROC-AUC by only 0.25 pp. Switching to single-resolution hashing cuts top- T recall to 91% and lowers Products ROC-AUC by 0.7 pp. Robustness Under Gaussian feature noise $\sigma = 0.1$, Products ROC-AUC drops by 1.3 pp for HASHPIPE-GAT versus 1.6 pp for HiPPO-GAT. Zero-shot transfer from Products to Friendster maintains 94% of original accuracy across all methods. Limitations HASHPIPE-GAT assumes largely static topology; highly dynamic graphs may require explicit cache invalidation. Although sketch noise is unbiased, it could interact with calibration in downstream tasks. The SimHash thresholds τ require minimal tuning across datasets.

7 Conclusion

HASHPIPE-GAT unifies historical edge caches, multi-resolution hashing and sketch-based dot-product approximation into a single GPU-friendly pipeline that reduces expected per-node complexity to $O(|C| + \log \log d + T)$. Experiments on graphs containing up to 1.8 billion edges demonstrate $\geq 1.5\times$ speed-ups over the best prior efficient kernel, parity or modest accuracy gains, and the first successful 12-layer full-batch training run on OGB-Papers100M within commodity GPU memory. Micro-studies validate the theoretical (ϵ, δ) guarantees, while hub-stress tests confirm a $30\times$ reduction in per-hub cost. Future work will explore adaptive cache sizing, automatic threshold tuning and

193 extensions to dynamic graphs. Combining HASHPIPE-GAT with graph sparsifiers or upcoming
194 sparse tensor cores promises further efficiency gains and advances the broader agenda of time-adaptive
195 attention for large-scale graph learning.

196 **References**